

A comparison of PostgreSQL, MariaDB and SQLite

Attribute	PostgreSQL	MariaDB	SQLite
Key features	Advanced data integrity, concurrency control, robust transaction support, scalability	Compatibility with MySQL, improved performance, high availability, scalability	Lightweight, serverless, embedded architecture, small memory footprint
Use cases	High traffic applications, data warehousing, geospatial data and GIS applications	Web applications, content management systems, e-commerce platforms, data replication, clustering	Mobile applications, IoT devices, local data storage, caching, prototyping, development environments
Performance	Efficient handling of complex queries and large data volumes	Good performance, especially for compatibility with MySQL	Suitable for low-resource environments, simplicity over high performance
Scalability and high availability	Options for horizontal and vertical scaling, replication, and clustering	Options for scaling horizontally and vertically, replication, and clustering	Not designed for high scalability or distributed architectures
Data types and querying capabilities	Extensive support for various data types, custom types, and complex queries	Good support for data types and querying capabilities	Limited set of data types and focused on simplicity
Community and ecosystem support	Strong and active open source community, wide range of extensions and integrations	Strong and active open source community, wide range of extensions and integrations	Smaller community compared to PostgreSQL and MariaDB, essential support available
Real-time scenario/example	Geospatial data analysis platform requiring advanced spatial querying capabilities and handling large volumes of location-based data	CMS platform with high availability, supporting concurrent user requests and online transactions	Mobile app requiring a lightweight, serverless database for caching and local data storage

horizontally and vertically, ensuring high availability and fault tolerance. They can handle large data sets, and support replication and clustering mechanisms. In contrast, SQLite is not designed for high scalability or distributed architectures. It is more suited for single-user or small-scale applications that do not require extensive scalability.

Data types and querying capabilities: PostgreSQL offers extensive support for various data types, including custom types, and allows complex queries, making it suitable for applications with diverse data requirements. MariaDB provides good support for data types and a wide range of querying capabilities, although it may not be as extensive as PostgreSQL in certain areas. SQLite, on the other hand, supports a limited

set of data types and is primarily focused on simplicity and ease of use. It is suitable for applications with straightforward data models and basic querying needs.

Community and ecosystem support: Both PostgreSQL and MariaDB have strong and active open source communities, providing continuous support, bug fixes, and updates. They also have a wide range of available extensions, plugins, and integrations, allowing developers to enhance their applications. PostgreSQL has a robust ecosystem and a thriving community that contributes to its continuous improvement. SQLite, although it has a smaller community compared to PostgreSQL and MariaDB, still has a dedicated user base and offers essential support for its users.

Examples of real-time scenarios

Here are some practical examples of which of these database management systems are best suited to different real-time scenarios.

Scenario 1: High-traffic social media platform

Ideal database: PostgreSQL

Explanation: A high-traffic social media platform with millions of users requires a robust database system to handle concurrent user interactions, complex relationships, and extensive data. PostgreSQL is the ideal choice due to its advanced data integrity and concurrency control mechanisms. It can effectively manage the high volume of user-generated content and handle complex queries efficiently. Additionally, PostgreSQL's scalability